

蓝桥杯30天算法冲刺集训



Day-18 最近公共祖先 (LCA)

定义

我们之前已经了解到“祖先”的概念，若两个节点的祖先相同，则叫做两个节点的 **公共祖先**。

最近公共祖先 (Lowest Common Ancestor)，简称 **LCA**，两个节点的最近公共祖先，就是这两个点的公共祖先里面，距离两个节点最近的公共祖先（离根节点最远）。

性质

为了方便，我们记某点集 $S = v_1, v_2, \dots, v_n$ 的最近公共祖先为 $\text{LCA}(v_1, v_2, \dots, v_n)$ 或 $\text{LCA}(S)$ 。

1. 两个节点的 LCA 有且只有一个，并且一定时两个节点到根的路径中重复部分最下端的点。
2. $\text{LCA}(u) = u$;
3. u 是 v 的祖先，当且仅当 $\text{LCA}(u, v) = u$;
4. 如果 u 不为 v 的祖先并且 v 不为 u 的祖先，那么 u, v 分别处于 $\text{LCA}(u, v)$ 的两棵不同子树中；
5. 前序遍历中， $\text{LCA}(S)$ 出现在所有 S 中元素之前，后序遍历中 $\text{LCA}(S)$ 则出现在所有 S 中元素之后；
6. 两点集并的最近公共祖先为两点集分别的最近公共祖先的最近公共祖先，即 $\text{LCA}(A \cup B) = \text{LCA}(\text{LCA}(A), \text{LCA}(B))$;
7. 两点的最近公共祖先必定处在树上两点间的最短路上；
8. $d(u, v) = h(u) + h(v) - 2h(\text{LCA}(u, v))$ ，其中 d 是树上两点间的距离， h 代表某点到树根的距离。

求法

方法一、朴素算法（暴力跳）

可以每次找深度比较大的那个点，让它向上跳。显然在树上，这两个点最后一定会相遇，相遇的位置就是要求的 LCA。

或者先向上调整深度较大的点，令他们深度相同，然后再共同向上跳转，最后也一定会相遇。

朴素算法预处理时需要 dfs 整棵树，时间复杂度为 $O(n)$ ，**单次查询** 时间复杂度为 $O(n)$ 。但由于随机树高为 $O(\log n)$ ，所以朴素算法在随机树上的单次查询时间复杂度为 $O(\log n)$ 。

参考代码：

```
#include <bits/stdc++.h>
using namespace std;
const int N = 5e5+5;
int n, m, s, x, y, fa[N], d[N];
int h[N], e[2*N], nxt[2*N], cnt;
void add(int x, int y)
{
    nxt[++cnt] = h[x];
    e[cnt] = y;
    h[x] = cnt;
}

// 求每个节点的深度和 fa
void dfs(int k, int f)
{
    fa[k] = f; d[k] = d[f]+1;
    for(int i = h[k]; i; i=nxt[i])
    {
        if(e[i] != f) dfs(e[i], k);
    }
}

// 方法一、每次让深度更大的节点往上跳
```

```

int lca(int x, int y)
{
    while(x!=y) {
        if(d[x] > d[y]) x = fa[x];
        else y = fa[y];
    }
    return x;
}

```

// 方法二、先向上调整深度较大的点，令他们深度相同，然后再共同向上跳转

```

int lca(int x, int y)
{
    // 把 x 移动到和 y 同一高度
    if(d[x] < d[y]) swap(x, y);
    while(d[x] > d[y]) x=fa[x];
    // 开始同步往上跳
    while(x!=y) {
        x = fa[x];
        y = fa[y];
    }
    return x;
}

```

```

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m >> s;
    for(int i = 1; i < n; i++)
    {
        cin >> x >> y;
        add(x, y); add(y, x);
    }
    // dfs 预处理获取深度和 fa 数组
    dfs(s, 1);
    for(int i = 1; i <= m; i++)
    {
        cin >> x >> y;
        cout << lca(x, y) << '\n';
    }
}

```

```
}  
    return 0;  
}
```

采用朴素算法求 LCA 的时间复杂度与两点间的距离有关，极限情况可达到 $O(n)$ 。虽然该算法时间复杂度较高，但该算法也是有一定的应用价值的：

1. 它实现简单，可在算法竞赛快速和正确的程序对拍。
2. 随机产生的树的高度期望是 $O(\log n)$ 级别的，有些时候树是随机的或者树的深度不大，我们也可以使用该算法。
3. 这个算法允许树动态改变。我们只需要知道每个点的父亲和深度，就可以方便地求出两个点的 LCA（当然如果更进一步地我们不知道点的深度，也可以先从一个点走到根，把路径上的点都标记了，再从另一个点向根走，走到标记的点为止）。另一个可以处理动态情况的数据结构是 **动态树**，但它实现复杂而且常数因子较大。所以在这个特定情况下，该算法具有不可替代的作用。

方法二、倍增算法

本算法是对算法一中一步步走的改进，核心实质是让两个结点 **每次向上走 2 的幂次步**，具体操作如下：

第一步，求出倍增数组：首先开一个 $n \times \log n$ 的数组，比如 $fa[n][\log n]$ ，其中 $fa[i][j]$ 表示 i 节点的第 2^j 个父亲。 $fa[i][j]$ 为结点 i 向上走 2^j 步后能走到的结点。

我们规定根结点的父亲是它自己，这样根结点往上走还是在根结点。

对于 $j = 0$ ， $fa[i][j]$ 就是结点 i 的父亲。

对于 $j > 0$ ， $fa[i][j]$ 等于 $fa[fa[i][j-1]][j-1]$ （即结点 i 往上走 2^{j-1} 步后再往上走 2^{j-1} 步）。

我们通过一遍从根节点开始的 dfs 预处理出 fa 数组和深度数组 d （可以方便地一起处理出来，后面要用到）。

第二步，把两个点移到同一深度：假设要求 $LCA(x, y)$ ，不失一般性，令 $d[x] \geq d[y]$ （ x 的深度大于等于 y 的深度，否则我们交换 x, y ），先让 x 往上走 $d[x] - d[y]$ 步。我们对这个差进行二进制拆分，就可以通过倍增数组往上走 2 的幂次步（即对于二进制为 1 的第 i 位。要往上走 2^i 步，即调用 $x = fa[x][i]$ ），那么可以在 $O(\log_2 n)$ 的时间复杂度内到达目标深度。或者说，类似的，我们也可以从大往小扫描 i ，一直尝试到 0（包括 0），

如果每次 $fa[x][i]$ 深度不小于 y ，我们就跳 x 。两种做法效果是一样的，读者可以根据自己的喜好选择。

第三步，求出 LCA：假设 x 与 y 向上走最小的 L 步后是同一结点，也就意味着， x 与 y 向上走最大的 $L - 1$ 步，也是不同的结点。我们可以从大到小枚举往上走 2^i 步，如果当前 x 与 y 向上走 2^i 步后为同一点，则停止，否则一起往上走。这样，我们就能在 $\log n$ 的时间复杂度内使 x 与 y 都向上走 $L - 1$ 步。根据倍增数组是 2 的幂次这个特性，这样的做法可以看成是一个通过 **二分法** 来求解走最大 $L - 1$ 步的过程。用这样的方法从大到小决策 $\log n$ 次，直到完全不能向上走了为止，我们再让 x 与 y 各向上走一步，则为 $LCA(x, y)$ 。

倍增算法的预处理时间复杂度为 $O(n \log n)$ ，**单次查询** 时间复杂度为 $O(\log n)$ 。

另外倍增算法可以通过交换 `fa` 数组的两维使较小维放在前面。这样可以减少 cache miss 次数，提高程序效率。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 5e5+5;
int n, m, s, x, y, fa[N][20], d[N], logn[N];
int h[N], e[2*N], nxt[2*N], cnt;
void add(int x, int y)
{
    nxt[++cnt] = h[x];
    e[cnt] = y;
    h[x] = cnt;
}

// 求每个节点的深度和 fa
void dfs(int u, int f)
{
    fa[u][0] = f; d[u] = d[f]+1;
    // 二分法计算 u 的第 2^i 个父结点
    // 是 u 的第 2^{i-1} 个祖先的第 2^{i-1} 个祖先
    for(int i = 1; i <= logn[d[u]]; i++)
        fa[u][i] = fa[fa[u][i-1]][i-1];
    for(int i = h[u]; i; i=nxt[i])
    {
```

```

        if(e[i] != f) dfs(e[i], u);
    }
}

// 先向上调整深度较大的点，令他们深度相同，然后再共同向上跳转
int lca(int x, int y)
{
    // 把 x 移动到和 y 同一高度
    if(d[x] < d[y]) swap(x, y);
    while(d[x] > d[y]) x=fa[x][logn[d[x]-d[y]]];
    if(x == y) return x;
    // 开始同步往上跳 2^i 步
    for(int i = logn[d[x]]; ~i; i--) {
        // 只要往上跳 2^i 步父节点还不相同，就继续上跳
        if(fa[x][i] != fa[y][i])
            x = fa[x][i], y = fa[y][i];
    }
    return fa[x][0]; // 第一个父结点就是 LCA
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m >> s;
    for(int i = 1; i < n; i++)
    {
        cin >> x >> y;
        add(x, y); add(y, x);
    }
    // 预处理 log_2 数组 logn[i]
    for(int i = 2; i <= n; i++)
        logn[i] = logn[i/2] + 1;
    // dfs 预处理获取深度和 fa 数组
    dfs(s, 0);
    for(int i = 1; i <= m; i++)
    {
        cin >> x >> y;
        cout << lca(x, y) << '\n';
    }
}

```

```
    }  
    return 0;  
}
```

蓝桥杯真题

机房（蓝桥杯C/C++2022B组国赛）

练习LCA的模板题。

这里我们算的树上路径的点权的和，所以我们直接套用下面的LCA的性质即可：

$d(u, v) = h(u) + h(v) - 2h(\text{LCA}(u, v))$ ，其中 d 是树上两点间的距离， h 代表某点到树根的距离。

注意我们求的是点权和，所以最后还要再加一个 $w[\text{LCA}(u, v)]$

```
#include <cstring>  
#include <iostream>  
#include <algorithm>  
#include <queue>  
  
using namespace std;  
const int N=100010,M=N*2;  
  
int depth[N],f[N][21];  
int h[N],ne[M],e[M],w[N],dist[N];  
bool st[N];  
int n,m,idx;  
  
void add(int a,int b)  
{  
    e[++idx]=b,ne[idx]=h[a],h[a]=idx,w[a]++;  
}  
  
void bfs(int root)  
{  
    memset(depth,0x3f,sizeof depth);
```

```

depth[0]=0,depth[root]=1;
queue<int> q;
q.push(root);
while(q.size())
{
    int t=q.front();
    q.pop();
    for(int i=h[t];i;i=ne[i])
    {
        int j=e[i];
        if(depth[j]>depth[t]+1)
        {
            depth[j]=depth[t]+1;
            q.push(j);
            f[j][0]=t;
            dist[j]=dist[t]+w[j];
            for(int k=1;k<=20;k++)
                f[j][k]=f[f[j][k-1]][k-1];
        }
    }
}

int lca(int a,int b)
{
    if(depth[a]<depth[b]) swap(a,b);
    int ans=0;
    for(int k=20;k>=0;k--)
    {
        if(depth[f[a][k]]>=depth[b])
            a=f[a][k];
    }
    if(a==b) return a;
    for(int k=20;k>=0;k--)
    {
        if(f[a][k]!=f[b][k])
        {
            a=f[a][k];
            b=f[b][k];
        }
    }
}

```

```

    }
}
return f[a][0];
}
int main()
{
    scanf("%d%d",&n,&m);
    for(int i=1;i<n;i++)
    {
        int a,b;
        scanf("%d%d",&a,&b);
        add(a,b),add(b,a);
    }
    dist[1]=w[1];
    bfs(1);
    while(m--)
    {
        int a,b;
        scanf("%d%d",&a,&b);
        int anc=lca(a,b);
        cout<<dist[a]+dist[b]-dist[anc]*2+w[anc]<<'\\n'

    }
    return 0;
}

```

版本分支（蓝桥杯Java2018B组国赛）

比上面的题还简单的LCA题，同学们是不是发现了蓝桥杯国赛考LCA都考的非常的简单，你只要能背过或者能理解的敲出LCA的模板，就能拿到一个题的满分。

这个题目直接如下判断即可：

```

if(lca(x,y)==x) cout<<"YES"<<endl;
else cout<<"NO"<<endl;

```

因为如果一个节点是另一个节点祖先，他们的最近公共祖先肯定就是这个节点了。

```

#include<bits/stdc++.h>
using namespace std;
const int N=1e5+2;
int n,q,fa[N][20],dep[N];
vector<int> g[N];
void dfs(int u,int p,int d)
{
    fa[u][0]=p;
    dep[u]=d;
    int sz=g[u].size();
    for (int i=0;i<sz;i++)
    {
        int v=g[u][i];
        if(v==p) continue;
        dfs(v,u,d+1);
    }
}
int lca(int x,int y)
{
    if(dep[x]<dep[y]) swap(x,y);
    for (int i=log2(dep[x]-dep[y]);i>=0;i--)
    {
        if(1<=i<=dep[x]-dep[y]) x=fa[x][i];
    }
    if(x==y) return x;
    for (int i=log2(dep[x]);i>=0;i--)
    {
        if(fa[x][i]!=fa[y][i])
        {
            x=fa[x][i];
            y=fa[y][i];
        }
    }
    return fa[x][0];
}
int main()
{
    cin>>n>>q;
    for (int i=1;i<=n-1;i++)

```

```

{
    int u,v;
    cin>>u>>v;
    g[u].push_back(v);
    g[v].push_back(u);
}
dfs(1,1,0);
for (int i=1;i<20;i++)
    for (int k=1;k<=n;k++)
        fa[k][i]=fa[fa[k][i-1]][i-1];
for (int i=1;i<=q;i++)
{
    int x,y;
    cin>>x>>y;
    if(lca(x,y)==x)
        cout<<"YES"<<endl;
    else
        cout<<"NO"<<endl;
}
return 0;
}

```

习题

冷战

分析

这道题第一眼看是并查集，对吧。但是不能直接用并查集做，因为会**超时**。我们可以将并查集和 LCA 联用起来。

首先因为我们只要两点相连通就行了。为避免成环，运用并查集判断连点是否连通。

对于找两个点最早是在加入第几条边连通，因为树上两个点间仅有一条简单路径，路径上的所有边中最后一条修建的边就是答案。

所以我们可以将即输入的边的序号当做权值，用一个数组 $val[i]$ 存权值，表示点 i 到他父亲的权值，那么每次询问的答案就是两点间路径上的边权的最大值。这个可以用 LCA 来解决。

赋权值操作可以放在启发式合并的时候进行。因为我们要寻找路径上所有边边权的最值，所以这里的并查集**不能用路径压缩**，不然路径上的边权信息就会丢失。又因为我们后边要用 $fa[i]$ 来尽心朴素 LCA，所以要用**启发式合并**，来提高算法运行效率。

然后在遇到要输出的时候，先用并查集判断两点是否在同一颗树上，如果在，就跑 LCA；否则，输出 0。

在跑 LCA 前，首先将每个点的深度预处理一下，这里不要暴力全部计算所有点的深度，否则效率比较低，容易超时。只需要分别自下而上递归遍历两个节点到根的路径上的所有节点，同时计算深度：

```
// 自下而上，递归计算深度
int depth(int x) {
    if(x == fa[x]) return d[x] = 1;
    return d[x] = depth(fa[x]) + 1;
}
```

```
#include <bits/stdc++.h>
using namespace std;

const int N = 5e5+5;
int n, m, u, v, op, ans;
// d[i]: 深度
// val[i]: i节点和其父节点连边的权值（是第几条边）
// h[i]: 并查集森林中 以 i 为根的树的高度，用于启发式合并
int fa[N], val[N], h[N], d[N], cnt;
// 找祖先，注意，不能做路径压缩
int find(int x) {
    if(fa[x] == x) return x;
    return find(fa[x]);
}
// 启发式合并
void merge(int x, int y) {
    int fx = find(x), fy = find(y);
    if(fx == fy) return;
```

```

    if(h[fx] > h[fy]) swap(fx, fy);
    if(h[fx] == h[fy]) h[fy]++;
    fa[fx] = fy;    // fx -> fy
    val[fx] = cnt;
}

// 自下而上，递归计算深度
int depth(int x) {
    if(x == fa[x]) return d[x] = 1;
    return d[x] = depth(fa[x]) + 1;
}

// 查询
int query(int x, int y) {
    int fx = find(x), fy = find(y);
    if(fx != fy) return 0;

    // 因为要查询路径上所有边，所以不能倍增，只能朴素暴力 LCA
    int res = 0;
    depth(x); depth(y); // 求出路径上节点深度
    // 移动到同一层，找路径上边权最大值更新 最早连通的时间
    if(d[x] < d[y]) swap(x, y);
    while(d[x] > d[y]) res = max(res, val[x]), x = fa[x];
    // 同步上跳，找路径上边权最大值更新 最早连通的时间
    while(x != y) {
        res = max(res, val[x]);
        res = max(res, val[y]);
        x = fa[x];
        y = fa[y];
    }
    return res;
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    for(int i = 1; i <= n; i++) fa[i] = i; // 初始化并查集
    while(m--) {
        cin >> op >> u >> v;
        u ^= ans; v ^= ans;    // 异或上一次询问的答案
    }
}

```

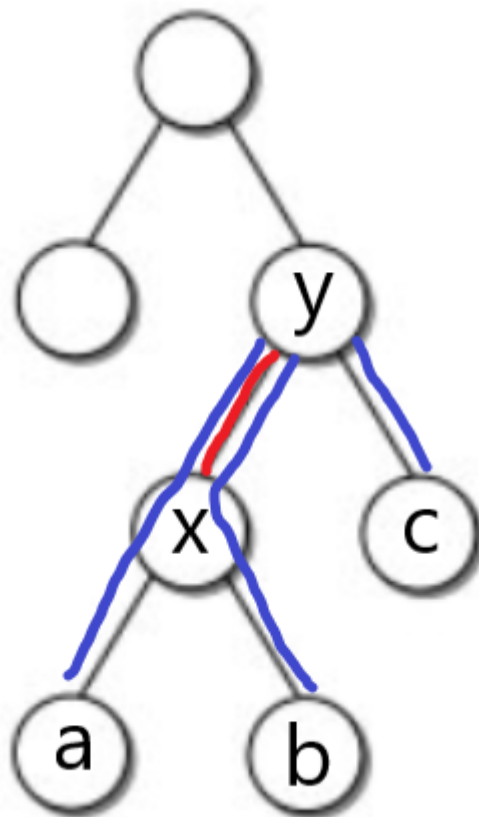
```

    if(op == 1) {
        ans = query(u, v);
        cout << ans << '\n';
    } else {
        cnt++; // 连接了一条边
        merge(u, v);
    }
}
return 0;
}

```

聚会

倍增求lca



以上图为例，我们发现三个点两两求lca，必定有两个lca是重复的
 $lca(a, c) = y, lca(b, c) = y$

换一种说法，对于任意的三个点 a, b, c ，必定存在一个点 y ，使得
 $lca(a, c) = y$ 且 $lca(b, c) = y$

(以上图为例, a, b, c可交换)

同时, 必定存在一个点 x 使得它到三个点的路径没有重叠部分。

发现了这些性质这个题就比较显然了

x 就是两个lca中深度较深的那个, 求出三个点到它的距离之和即可, 距离之和等于蓝边之和减去红边

时间复杂度 $O(m\log n)$

```
#include<cstdio>
#include<cstring>
#include<algorithm>

using namespace std;

const int N = 5e5 + 10, M = N * 2;

int h[N], e[M], ne[M], idx;
void add(int a, int b)
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++;
}

int n, m;
int dep[N], f[N][20];
void dfs(int u, int fa)
{
    dep[u] = dep[fa] + 1, f[u][0] = fa;
    for(int i = 1; i < 20; i ++) f[u][i] = f[f[u][i - 1]][i - 1];

    for(int i = h[u]; ~i; i = ne[i])
    {
        int j = e[i];
        if(j == fa) continue;
        dfs(j, u);
    }
}
```

```

int lca(int a, int b)
{
    if(dep[a] < dep[b]) swap(a, b);

    for(int i = 19; i >= 0; i --)
        if(dep[f[a][i]] >= dep[b])
            a = f[a][i];

    if(a == b) return a;

    for(int i = 19; i >= 0; i --)
        if(f[a][i] != f[b][i])
            a = f[a][i], b = f[b][i];
    return f[a][0];
}

int main()
{
    memset(h, -1, sizeof h);
    scanf("%d%d", &n, &m);
    for(int i = 1; i < n; i ++)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        add(a, b), add(b, a);
    }

    dfs(1, 0);

    while(m --)
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);

        int x = lca(a, b), y = lca(a, c), z = lca(b, c);

        if(x == y) swap(y, z);           // 让z成为那个重复
    }
}

```

的

```
        if(dep[x] < dep[y]) swap(x, y);    // 让x成为深度较大
    的

        int sum = dep[a] + dep[b] + dep[c] - dep[y] * 3 -
    (dep[x] - dep[y]);
        printf("%d %d\n", x, sum);
    }
    return 0;
}
```